

On the Optimal Source Key Size of Secure Gradient Coding

Yang Zhou*, Wenbo Huang*, Kai Wan*, Robert Caiming Qiu*

*Huazhong University of Science and Technology, 430074 Wuhan, China,
{hust_zhou, eric_huang, kai_wan, caiming}@hust.edu.cn

Abstract—Gradient coding enables a user node to efficiently aggregate gradients computed by server nodes from local datasets, achieving low communication costs while ensuring resilience against straggling servers. This paper considers the secure gradient coding problem, where a user aims to compute the sum of the gradients from K datasets with the assistance of N distributed servers. The user is required to recover the sum of gradients from the transmissions of any N_r servers, with each dataset assigned to $N - N_r + m$ servers. The security constraint guarantees that even if the user receives transmissions from all servers, no additional information about the datasets can be obtained beyond the sum of gradients. It has been shown in the literature that the security constraint does not increase the optimal communication cost of the gradient coding problem, provided that enough source keys are shared among the servers. However, the minimum required source key size to ensure security while achieving the optimal communication cost has been studied only for the case $m = 1$. In this paper, we focus on the more general case $m \geq 1$ and aim to characterize the minimum required source key size for this purpose. A new information-theoretic converse bound on the source key size and a novel achievable scheme with smartly designed assignments are proposed. Our proposed scheme outperforms the optimal scheme based on the widely used cyclic data assignment and coincides with the converse bound under specific system parameters.

I. INTRODUCTION

Secure aggregation is a well-established topic in distributed computation that addresses the multiparty computation problem in which the goal is to compute a sum over multiple parties without revealing individual updates, not even to the aggregator [1]. This primitive has already seen widespread deployment in privacy-sensitive applications, ranging from on-device next-word prediction in mobile keyboards [2] to cross-institutional medical-model training [3]. The concept of secret sharing was first introduced in [4], where a secret is divided into multiple shares so that only participants meeting a predefined threshold can reconstruct the secret. The complexity of verifiable secret sharing and its extension to multiparty computation was further investigated in [5]–[8].

In this paper, we consider the secure aggregation in gradient coding, called secure gradient coding. The gradient coding problem was first proposed in [9]. In a general gradient coding model, a user wants to compute a sum of gradients on K datasets by N servers. During the data assignment phase, each dataset is assigned to $M = N - N_r + m$ servers. During the computing phase, each server first computes gradients of the assigned datasets and then sends a coded message on these gradients to the user. During the decoding phase,

the user should recover the computation task result using the transmissions from any N_r servers. The objective is to minimize the number of transmissions (i.e., communication cost). In [10], the optimal communication cost under linear encoding was characterized. Furthermore, the authors in [11] consider heterogeneous (and arbitrarily fixed) data assignment, where each dataset may be assigned to a different number of servers, and characterizes the optimal communication cost under linear encoding.

The secure aggregation model for gradient coding was first proposed in [12].¹ In this model, a key distribution server assigns a shared key to all servers to ensure security. It was proven in [12] that the security constraint will not increase the optimal communication cost of the gradient coding problem, if enough source keys are shared among the servers. Then for the case $m = 1$ (i.e., when M is minimum to tolerate $N - N_r$ stragglers), an information-theoretic lower bound on the source key size and several secure distributed coded computation schemes for specific data distribution patterns (e.g., cyclic, repetitive, and mixed) were proposed in [12], while achieving the optimal communication cost with different requirement on the source key size.

Recently, secure aggregation for federated learning has also gained significant attention [13]–[17]. In secure aggregation for federated learning, each distributed computing node holds its local data to compute the gradient; thus, secure aggregation against stragglers (or user dropouts) requires two-round transmissions. In contrast, in the gradient coding problem, an additional data assignment phase for servers exists. Because of the redundancy in data assignment, one-round transmission is enough for secure aggregation.

Main Contributions: In this paper, we aim to characterize the minimum source key size for the case $m > 1$ while achieving the optimal communication cost under linear encoding. Our main contributions are as follows.

- For a given data assignment, we provide an information-theoretic converse bound on the source key size, while achieving the optimal communication cost under linear encoding.
- To reduce the source key size, we introduce a secure gradient coding scheme with a new data assignment

¹The secure distributed linearly separable computation was formulated in [12]. When the user only requests one linear combination of the gradients (or input vectors), the problem reduces to secure gradient coding.

strategy. This scheme outperforms the widely used cyclic assignment scheme and can achieve the converse bound under some system parameters.

Notation Convention: Calligraphic symbols denote sets, bold lower-case letters denote vectors, bold upper-case letters denote matrices, and sans-serif symbols denote system parameters. We use $|\cdot|$ to represent the cardinality of a set or the length of a vector; $[a : b] := \{a, a+1, \dots, b\}$; $[n] := [1 : n]$; $\mathbb{F}_q$ represents a finite field with order q ; $\mathbf{M}^T$ and $\mathbf{M}^{-1}$ represent the transpose and the inverse of matrix $\mathbf{M}$, respectively; the matrix $[a; b]$ is written in a Matlab form, representing $[a, b]^T$; $(\mathbf{M})_{m \times n}$ represents the dimension of matrix $\mathbf{M}$ is $m \times n$; $\text{Mod}(b, a)$ represents the modulo operation on b with integer divisor a and in this paper we let $\text{Mod}(b, a) \in \{1, \dots, a\}$ (i.e., we let $\text{Mod}(b, a) = a$ if a divides b).

II. SYSTEM MODEL

This section describes the (K, N, N_r, m) secure gradient coding problem. We consider a framework in which a user, assisted by N servers, aims to compute the sum of gradients derived from K independent datasets $D_1, \dots, D_K$, where each gradient represented as a string of L symbols over a sufficiently large alphabet. Compared to distributed computing without considering secure aggregation, there is another trusted server to distribute keys to the computing servers. It generates a set of random variables $\mathcal{Q} = \{Q_1, \dots, Q_N\}$ on $\mathbb{F}_q$ independent of the datasets,

$$I(Q_1, \dots, Q_N; D_1, \dots, D_K) = 0. \quad (1)$$

The source key size η measures the total amount of randomness among all keys, i.e.,

$$\eta = H(Q_1, \dots, Q_N)/L. \quad (2)$$

The secure gradient coding framework comprises three phases: *data assignment*, *computing*, and *decoding*.

Data assignment phase. We define N_r as the number of minimum available servers, and the identity of the surviving servers is not known until the decoding phase. The data center assigns each dataset to $M = N - N_r + m$ servers, where m is the computation cost factor (which is also called communication reduction factor in [10]). The index set of datasets assigned to server $n \in [N]$ is denoted as $\mathcal{Z}_n \in [K]$. In addition, the trusted key server allocates the key Q_i to each server $i \in [N]$.

Computing phase. Each server $n \in [N]$ first computes the message $g_k = \nabla(D_k)$ for each $k \in \mathcal{Z}_n$. Then it sends the coded message to the user,

$$X_n = \psi_n(\{g_k : k \in \mathcal{Z}_n\}, Q_n), \quad (3)$$

where ψ_n is a function of messages $\{g_k : k \in \mathcal{Z}_n\}$,

$$\psi_n : \mathbb{F}_q^{|\mathcal{Z}_n|L} \times |\mathcal{Q}| \rightarrow \mathbb{F}_q^{T_n}, \quad (4)$$

and T_n represents the length of X_n . m is the computation cost factor because in a distributed system, the complexity of computing the linear combinations of the gradients is much lower than computing the gradients.

Decoding phase. The user should be able to use coded messages from any N_r servers to decode the computation tasks. Thus, for each subset of servers $\mathcal{A} \subseteq [N]$ where $|\mathcal{A}| = N_r$, by defining $X_{\mathcal{A}} := (X_n : n \in \mathcal{A})$, it should hold that

$$H(g_1 + \dots + g_K | X_{\mathcal{A}}) = 0. \quad (5)$$

We define

$$R := \max_{\mathcal{A} \subseteq [N]: |\mathcal{A}| = N_r} \frac{\sum_{n \in \mathcal{A}} T_n}{L} \quad (6)$$

as the communication cost, which represents the maximum normalized number of symbols received from any N_r servers to recover the computational task.

The secure aggregation protocol imposes that

$$I(g_1, \dots, g_K; X_{[N]} | g_1 + \dots + g_K) = 0, \quad (7)$$

where $X_{[N]}$ presents all messages from N servers which may be received by the user. This equation ensures that the user cannot obtain any information about the datasets except for the computation task.

If the source key size is large enough, the optimal communication cost under linear encoding (i.e., ψ_n is a linear function), can be characterized by directly combining [10], [12].²

Theorem 1 ([10], [12]). *For the (K, N, N_r, m) secure gradient coding problem, the optimal communication cost under linear encoding is $\frac{N_r}{m}$.*

Our main objective in this paper is to minimize the source key size η while maintaining the optimal communication cost under linear encoding for the case $m \geq 1$.

III. MAIN RESULTS

We first introduce a novel converse bound on η for a fixed assignment, whose proof can be found in Appendix A.

Theorem 2. *For the (K, N, N_r, m) secure gradient coding problem, for a fixed assignment $\mathbf{Z} = (\mathcal{Z}_1, \dots, \mathcal{Z}_N)$, if there exists an ordered set of servers in $[N]$, denoted by $\mathbf{s} = (s_1, \dots, s_{|\mathbf{s}|})$, where each server contains at least one dataset that appears in the datasets of its preceding servers at most $m - 1$ times, this can be expressed as:*

$$\forall i \in [|\mathbf{s}|], \exists x \in \mathcal{Z}_{s_i} \text{ such that } \sum_{j=1}^{i-1} \delta(x \in \mathcal{Z}_{s_j}) \leq m - 1, \quad (8)$$

where $\delta(x \in \mathcal{Z}_{s_j})$ equals 1 if $x \in \mathcal{Z}_{s_j}$, and 0 otherwise.

Under the symmetric transmission and linear coding, it must hold that

$$\eta \geq \frac{|\mathbf{s}|}{m} - 1. \quad (9)$$

Theorem 2 indicates that minimizing source key size η is a combinatorial problem that depends on the data assignment.

²Without security, the authors in [10] characterized the optimal communication cost under linear encoding is N_r/m . In addition, [12, Theorem 1] showed that for any (K, N, N_r, m) non-secure gradient coding problem, the coding scheme can be made secure without increasing the communication cost.

Further, we can establish a min-max problem to characterize the minimum η^* as follows.

Corollary 1. *For the (K, N, N_r, m) secure gradient coding problem, we have*

$$\eta^* \geq \min_{\mathbf{Z}} \max_{\mathbf{s} \subseteq [N], \text{ subject to (8)}} \frac{|\mathbf{s}|}{m} - 1. \quad (10)$$

The essence of min-max problems lies in balancing extreme values across all chosen $\mathbf{s}$. In order to simplify the computation, we provide a converse bound on η^* , whose proof could be found in Appendix A.

Corollary 2. *For the (K, N, N_r, m) secure gradient coding problem, it must hold that*

$$\eta^* \geq \left\lceil \frac{mN}{N - N_r + m} \right\rceil - 1. \quad (11)$$

The proof could be found in Appendix B.

Next, we propose a novel secure gradient coding scheme that employs a new data assignment strategy, improving the commonly used cyclic assignment.

Theorem 3. *For the (K, N, N_r, m) secure gradient coding problem, the source key size $\eta = \frac{h(N, M)}{m} - 1$ is achievable, where the output of the function $h(\cdot, \cdot)$ is given by the recursive algorithm shown in Fig. 1 with the following properties:*

- When $N > 2M$, by Scheme 1 described in Section IV-A we have

$$h(N, M) = h(N - \lfloor N/M - 1 \rfloor M, M) + m \lfloor N/M - 1 \rfloor. \quad (12)$$

- When $1.5M \leq N < 2M$ and M is even, with $M \geq 2m$, by Scheme 2 described in Section IV-B, we have

$$h(N, M) = h\left(N - M, \frac{M}{2}\right) + m. \quad (13)$$

- When $1.5M \leq N < 2M$ and M is odd, with $M \geq 2m + 1$, by Scheme 3 described in Section IV-C we have

$$h(N, M) = N - \frac{3M - 1}{2} + 2m; \quad (14)$$

- When $M < N < 1.5M$, with $M \geq 2m$, by Scheme 4 described in Section IV-D we have

$$h(N, M) = h(M, 2M - N). \quad (15)$$

□

Remark 1 (The source key size in extreme cases). For the (K, N, N_r, m) secure gradient coding problem with $M = N - N_r + m$, and M divides N , the minimum source key size is

$$\eta^* = \frac{N}{M} - 1, \quad (16)$$

which matches the converse (2) and is achievable using a fractional repetition assignment.

Under the cyclic assignment constraint, the minimum source

key size required for optimal communication cost is

$$\eta_{\text{cyc}}^* = \frac{N_r}{m} - 1. \quad (17)$$

At the end of this section, we present numerical evaluations comparing the source key size in the converse in (11), in the cyclic assignment scheme, and the scheme from Theorem 3. The results demonstrate that the combined scheme requires a significantly smaller source key size than the cyclic assignment scheme and coincides with the converse at some points.

IV. NEW ACHIEVABLE SCHEMES FOR THEOREM 3

For any secure distributed linearly separable scheme, [12, Theorem 1] provides a transfer method from a non-secure scheme to a secure scheme while maintaining the communication cost. The required source key size is $\frac{\lambda}{m} - 1$, where λ represents the number of linearly independent messages transmitted by all the servers, i.e., $H(X_1, \dots, X_K)/L$. Hence, to design a secure gradient coding scheme achieving the optimal communication cost with source key size as small as possible, we can design a non-secure gradient coding scheme achieving the optimal communication cost while minimizing the number of linearly independent messages. Note that when $N|K$, we can transfer the problem into $K = N$ using [12, Remark 1]. Thus, we focus on minimizing λ in the non-secure scheme where $K = N$. To elucidate the coding scheme, we present an illustrative example in Scheme 3, where the general description is given in Appendix C.

We mainly solve two challenges introduced by the transition from $m = 1$ to $m > 1$: when dividing the servers into groups and assigning different datasets to different groups, datasets within the same group should be stored at least m times, we design a more complex structure; to reduce λ , we do not use the coding scheme in [11], but jointly design the coding scheme across different groups using transmitter Interference Alignment [18]–[20]. For details, readers can refer to Appendix C.

A. Scheme 1 for (12)

In this section, we consider the case where $N > 2M$. We partition servers and datasets into intervals of length M , i.e., $[(i-1)M + 1 : iM]$ for $i \in \left[\left\lfloor \frac{N}{M} - 1 \right\rfloor\right]$, and assign them via the fractional repetition strategy, yielding $m \left\lfloor \frac{N}{M} - 1 \right\rfloor$ linearly independent combinations. We then assign the remaining $N - \left\lfloor \frac{N}{M} - 1 \right\rfloor M$ datasets and servers, thereby reducing the original (N, M) problem to a smaller subproblem $(N - \left\lfloor \frac{N}{M} - 1 \right\rfloor M, M)$.

B. Scheme 2 for (13)

We consider the case $1.5M \leq N < 2M$ with even $M \geq 2m$. The servers are divided into three groups: those in $[1, \frac{M}{2}]$ (first group), those in $[\frac{M}{2} + 1, M]$ (second group), and the remaining $N - M$ servers (third group). The first two groups each provide m linearly independent linear combinations, while the third group employs the $(N - M, \frac{M}{2})$ scheme to recover $h(N - M, \frac{M}{2})$ combinations.

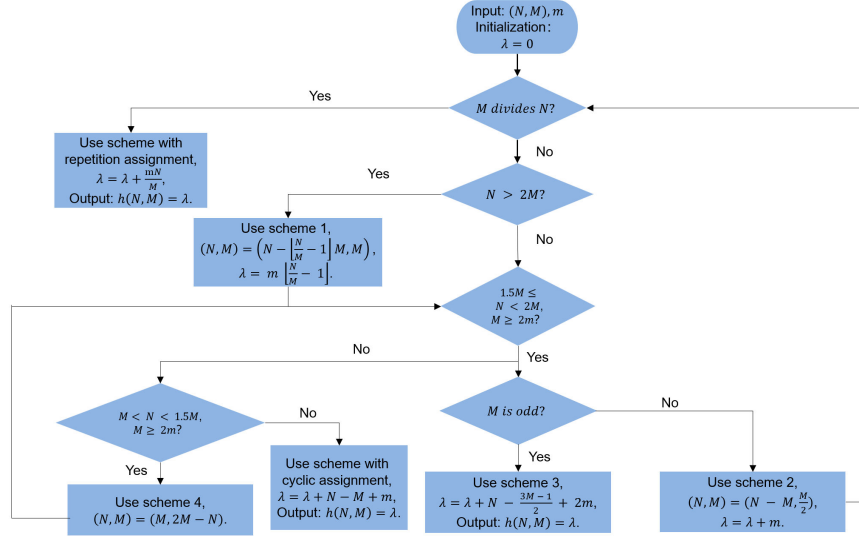


Fig. 1: Flow diagram of the combined scheme in Theorem 3.

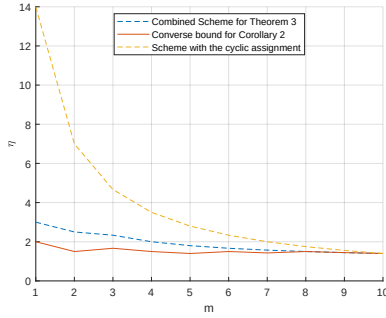


Fig. 2: Numerical evaluations for the secure gradient coding with $K = N = 24, M = 10$.

We first consider the data assignment for the servers in $[M]$, whose structure is as follows.

server	1	...	$\frac{M}{2}$	$\frac{M}{2} + 1$	...	M
dataset	D_1	...	D_1	D_1	...	D_1
	...	...	...	...	...	...
	D_y	...	D_y	D_y	...	D_y
	D_{y+1}	...	D_{y+1}	D_{M+1}	...	D_{M+1}
	...	...	...	...	...	...
	D_M	...	D_M	D_N	...	D_N

In this setup, we assign D_1 to D_y to all servers in $[M]$, while each D_k for $k \in [y+1 : N]$ is assigned to $\frac{M}{2}$ servers in $[M]$. We then allocate the remaining $N - M$ servers. Since $N - y = 2(N - M)$ datasets (from $[y+1 : N]$) must be distributed among $N - M$ servers, each dataset is assigned to $\frac{M}{2}$ servers, and each server receives M datasets.

To implement this, we divide the datasets in $[y+1 : N]$ into $\frac{N-y}{2} = N - M$ pairs $\mathcal{P}_i = \{y+i, M+i\}$ for $i \in [N-M]$. We then use the assignment phase of the $(N-M, \frac{M}{2})$ scheme, assigning each pair to $\frac{M}{2}$ servers, with each server receiving $\frac{M}{2}$ pairs.

C. Scheme 3 for (14)

We provide an example to illustrate the main idea.

Example 1. We consider the $(N, M) = (12, 7)$ non-secure problem, where $m = 2$.

Data assignment phase. We assign the datasets as follows.

server	1	2	3	4	5	6	7
dataset	D_1	D_1	D_1	D_1	D_1	D_1	D_1
	D_2	D_2	D_2	D_2	D_2	D_2	D_2
	D_3	D_3	D_3	D_3	D_3	D_3	D_3
	D_4	D_4	D_8	D_9	D_{10}	D_{11}	D_{12}
	D_5	D_5	D_9	D_{10}	D_{11}	D_{12}	D_8
	D_6	D_6	D_{10}	D_{11}	D_{12}	D_8	D_9
	D_7	D_7	D_{11}	D_{12}	D_8	D_9	D_{10}

8	9	10	11	12
D_4	D_4	D_4	D_4	D_4
D_5	D_5	D_5	D_5	D_5
D_6	D_6	D_6	D_6	D_6
D_7	D_7	D_7	D_7	D_7
D_8	D_9	D_{10}	D_{11}	D_{12}
D_9	D_{10}	D_{11}	D_{12}	D_8
D_{10}	D_{11}	D_{12}	D_8	D_9

Computing phase. To minimize communication cost, each message W_k , for $k \in [K]$, is divided into m equal-length sub-messages, denoted as $W_k = \{W_{k,j} : j \in [m]\}$, with each sub-message containing $\frac{L}{m} = \frac{L}{2}$ symbols in $\mathbb{F}_q$. Each server sends a linear combination of messages, allowing the user to recover $\mathbf{F}[W_{1,1}; \dots; W_{12,2}]$ from any $N_r = 7$ servers, where $\mathbf{F}$ is shown at the top of the page in (18). The design of $\mathbf{f}_5$ and $\mathbf{f}_6$ will be detailed later. We define that $[F_1; F_2; F_3; F_4; F_5; F_6] = \mathbf{F}[W_{1,1}; \dots; W_{12,2}]$.

Key step: We divide the servers into three groups based on the data assignment: servers 1 and 2 form the first group, those in $[3 : 7]$ form the second, and those in $[8 : 12]$ form

$$\mathbf{F} = \begin{bmatrix} \mathbf{f}_1 \\ \mathbf{f}_2 \\ \mathbf{f}_3 \\ \mathbf{f}_4 \\ \mathbf{f}_5 \\ \mathbf{f}_6 \end{bmatrix} = \begin{bmatrix} 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 0 & 0 & 0 & 2 & 2 & 2 & 2 & 1 & 1 & 1 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 2 & 2 & 2 & 2 & 1 & 1 & 1 & 1 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 2 & 4 & 2 & 4 & 2 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 2 & 1 & 3 & 4 & 2 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 2 & 2 & 4 & 4 & -2 & 0 & 0 & 0 & 0 & 0 & 0 & 3 & 3 & 1 & 1 & -5 & -5 \end{bmatrix} \quad (18)$$

the third. We require that the $m = 2$ linear combinations $(F_1 - F_3)$ and $(F_2 - F_4)$ be recovered from the first group. From the second group, we recover $(2F_1 - F_3)$, $(2F_2 - F_4)$, F_5 , and F_6 , which together form $N - \frac{3M-1}{2} + m = 4$ linear combinations, and from the third group, we recover F_3 , F_4 , F_5 , and F_6 , another 4 combinations. Hence, we can recover $N - \frac{3M-1}{2} + 2m = 6$ linear combinations in total, matching the result in (14). The following outlines the specific procedure.

We focus on servers 1 and 2, which share the same datasets and exclude those from [8:11]. server 1 computes

$$X_1 = (F_1 - F_3) + (F_2 - F_4),$$

and server 2 computes

$$X_2 = (F_1 - F_3) - (F_2 - F_4).$$

For servers handling [8:12], we design transmissions so that F_3, F_4, F_5 , and F_6 can be recovered from any four responses. Specifically, for $n \in [8:12]$, let server n compute

$$\mathbf{s}_n [\mathbf{f}_3; \mathbf{f}_4; \mathbf{f}_5; \mathbf{f}_6] [W_{1,1}; \dots; W_{12,2}],$$

We design $\mathbf{s}_n$ according to the following steps. We focus on the datasets in [8 : 12] under cyclic assignment, extracting the corresponding columns from the matrix to form a submatrix $\mathbf{F}'_1$.

Notice that server n cannot compute two datasets in [8 : 12]. We extract the corresponding columns from the matrix to form a submatrix $\mathbf{F}'_1$.

Our required $\mathbf{s}_n$ is the left null vector of $\mathbf{F}'_1$. There should be a linear combination of the columns in $\mathbf{F}'_1$ leading to one rank deficiency, with the form

$$\mathbf{F}'_1 \mathbf{e}_n^T = \mathbf{0}_{4 \times 1}, \quad (19)$$

for the first row of $\mathbf{F}'_1$, since $\begin{bmatrix} 1 & 1 \end{bmatrix} [1, -1]^T = \mathbf{0}_{1 \times 1}$, the corresponding elements in $\mathbf{e}_n$ should be a multiple of $(1, -1)$, and we choose a random multiple. Similarly, the same applies to the second row of $\mathbf{F}'_1$. Repeating the above steps, we obtain the matrix $\mathbf{E}$, where

$$\mathbf{E} = \begin{bmatrix} \mathbf{e}_1 \\ \vdots \\ \mathbf{e}_5 \end{bmatrix} = \begin{bmatrix} 0 & 0 & 0 & 1 & -1 & 0 & 0 & 0 & -1 & 1 \\ 2 & 0 & 0 & 0 & -2 & -1 & 0 & 0 & 0 & 1 \\ 1 & -1 & 0 & 0 & 0 & 2 & -2 & 0 & 0 & 0 \\ 0 & 1 & -1 & 0 & 0 & 0 & 1 & -1 & 0 & 0 \\ 0 & 0 & -1 & 1 & 0 & 0 & 0 & 2 & -2 & 0 \end{bmatrix} \quad (20)$$

By (19) and (20), we have

$$\mathbf{F}'_1 \mathbf{E}^T = \mathbf{0}_{4 \times 5}, \quad (21)$$

where $\mathbf{E}^T$ is 10×5 and full rank. Thus, its left null space contains 5 linearly independent vectors, spanning the rows of $\mathbf{F}'_1$. The first two rows lie in the null space of $\mathbf{E}^T$, and any two

vectors from the remaining three null space vectors complete $\mathbf{F}'_1$, given as:

$$\mathbf{F}'_1 = \begin{bmatrix} 1 & 1 & 1 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 1 & 1 & 1 & 1 & 1 \\ 2 & 4 & 2 & 4 & 2 & 2 & 1 & 3 & 4 & 2 & 2 \\ 2 & 2 & 4 & 4 & -2 & 3 & 3 & 1 & 1 & -5 & -5 \end{bmatrix}.$$

For $\mathbf{s}_n$, satisfying $\mathbf{s}_n \mathbf{F}'_1 = \mathbf{0}_{1 \times 4}$, the resulting matrix is:

$$\mathbf{S} = \begin{bmatrix} \mathbf{s}_1 \\ \vdots \\ \mathbf{s}_5 \end{bmatrix} = \begin{bmatrix} 2 & 11/4 & -3/4 & 1/4 \\ 4 & 4 & -2 & 0 \\ 2 & 3 & 0 & -1 \\ 8 & 16/3 & -4/3 & -4/3 \\ 6 & 3/2 & 0 & -3/2 \end{bmatrix}.$$

For $n \in [8, 12]$, server n computes

$$X_n = \mathbf{s}_n [F_3; F_4; F_5; F_6].$$

Next, servers in [3:7], excluding datasets from [4:7], ensure that responses from any 4 servers recover $(2F_1 - F_3)$, $(2F_2 - F_4)$, F_5 , and F_6 . For $n \in [3, 7]$, server n computes

$$X_n = \mathbf{s}_n [2\mathbf{f}_1 - \mathbf{f}_3; 2\mathbf{f}_2 - \mathbf{f}_4; \mathbf{f}_5; \mathbf{f}_6] [W_{1,1}; \dots; W_{12,2}].$$

For datasets in [8:12], cyclic assignment forms $\mathbf{F}'_2$, and it is evident that $\mathbf{F}'_2 = \mathbf{F}'_1$. Since datasets missing from server $n \in [3:7]$ are included in those missing from $n+5$, we have $\mathbf{S}' = \mathbf{S}$. Thus, server n computes

$$X_n = \mathbf{s}_n [2F_1 - F_3; 2F_2 - F_4; F_5; F_6].$$

In conclusion, under the condition $m = 2$, the number of totally transmitted linearly independent combinations is $h(12, 7) = 12 - 10 + 4 = 6$, which coincides with (14). Moreover, our calculations show that if

$$m(N - M - 1) \leq [(N - M)(m - 1) + 1] \left(N - \frac{3}{2}M - \frac{1}{2} \right),$$

transmitter Interference Alignment [18]–[20] is required.

D. Scheme 4 for (15)

In this section, we consider the case $M < N < 1.5M$ with $M \geq 2m$ and transform the (N, M) problem into $(M, 2M - N)$ by dividing the servers into two groups: those in $[M]$ and those in $[M + 1 : N]$. We assign $D_1, \dots, D_{N-M}$ to each server in $[M]$, and $D_{N-M+1}, \dots, D_N$ to each server in $[M + 1 : N]$. Thus, every server in $[M + 1 : N]$ holds M datasets, while each server in $[M]$ has $N - M$. Since every dataset in $[N - M + 1 : N]$ is assigned to $N - M$ servers, we further assign each D_k (for $k \in [N - M + 1 : N]$) to $2M - N$ additional servers in $[M]$, giving each such server $(2M - N)$ datasets. This allocation follows the assignment phase of the $(M, 2M - N)$ non-secure problem.

REFERENCES

- [1] K. Bonawitz, V. Ivanov, B. Kreuter, A. Marcedone, H. B. McMahan, S. Patel, D. Ramage, A. Segal, and K. Seth, "Practical secure aggregation for privacy-preserving machine learning," in *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*, ser. CCS '17. New York, NY, USA: Association for Computing Machinery, 2017, p. 1175–1191. [Online]. Available: <https://doi.org/10.1145/3133956.3133982>
- [2] —, "Practical secure aggregation for privacy-preserving machine learning," in *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*, ser. CCS '17. New York, NY, USA: Association for Computing Machinery, 2017, p. 1175–1191. [Online]. Available: <https://doi.org/10.1145/3133956.3133982>
- [3] R. Shokri and V. Shmatikov, "Privacy-preserving deep learning," in *Proceedings of the 22nd ACM SIGSAC Conference on Computer and Communications Security*, ser. CCS '15. New York, NY, USA: Association for Computing Machinery, 2015, p. 1310–1321. [Online]. Available: <https://doi.org/10.1145/2810103.2813687>
- [4] A. Shamir, "How to share a secret," *Commun. ACM*, vol. 22, no. 11, p. 612–613, Nov. 1979. [Online]. Available: <https://doi.org/10.1145/359168.359176>
- [5] T. P. Pedersen, "Non-interactive and information-theoretic secure verifiable secret sharing," in *Annual international cryptology conference*. Springer, 1991, pp. 129–140.
- [6] R. Cramer, I. Damgård, and S. Dziembowski, "On the complexity of verifiable secret sharing and multiparty computation," in *Proceedings of the thirty-second annual ACM symposium on Theory of computing*, 2000, pp. 325–334.
- [7] D. Chaum, C. Crépeau, and I. Damgård, "Multiparty unconditionally secure protocols," in *Proceedings of the Twentieth Annual ACM Symposium on Theory of Computing*, ser. STOC '88. New York, NY, USA: Association for Computing Machinery, 1988, p. 11–19. [Online]. Available: <https://doi.org/10.1145/62212.62214>
- [8] X. Yue, H. Wang, D. Jin, M. Li, and W. Jiang, "Healthcare data gateways: found healthcare intelligence on blockchain with novel privacy risk control," *Journal of medical systems*, vol. 40, pp. 1–8, 2016.
- [9] R. Tandon, Q. Lei, A. G. Dimakis, and N. Karampatziakis, "Gradient coding: Avoiding stragglers in distributed learning," in *Proceedings of the 34th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, D. Precup and Y. W. Teh, Eds., vol. 70. PMLR, 06–11 Aug 2017, pp. 3368–3376. [Online]. Available: <https://proceedings.mlr.press/v70/tdandon17a.html>
- [10] M. Ye and E. Abbe, "Communication-computation efficient gradient coding," in *International Conference on Machine Learning*. PMLR, 2018, pp. 5610–5619.
- [11] T. Jahani-Nezhad and M. A. Maddah-Ali, "Optimal communication-computation trade-off in heterogeneous gradient coding," *IEEE Journal on Selected Areas in Information Theory*, vol. 2, no. 3, pp. 1002–1011, 2021.
- [12] K. Wan, H. Sun, M. Ji, and G. Caire, "On secure distributed linearly separable computation," *IEEE Journal on Selected Areas in Communications*, vol. 40, no. 3, pp. 912–926, 2022.
- [13] H. Fereidooni, S. Marchal, M. Miettinen, A. Mirhoseini, H. Möllering, T. D. Nguyen, P. Rieger, A.-R. Sadeghi, T. Schneider, H. Yalame, and S. Zeitouni, "Safelearn: Secure aggregation for private federated learning," in *2021 IEEE Security and Privacy Workshops (SPW)*, 2021, pp. 56–62.
- [14] B. Choi, J.-y. Sohn, D.-J. Han, and J. Moon, "Communication-computation efficient secure aggregation for federated learning," *arXiv preprint arXiv:2012.05433*, 2020.
- [15] J. So, C. He, C.-S. Yang, S. Li, Q. Yu, R. E. Ali, B. Guler, and S. Avestimehr, "Lightsecagg: a lightweight and versatile design for secure aggregation in federated learning," in *Proceedings of Machine Learning and Systems*, D. Marculescu, Y. Chi, and C. Wu, Eds., vol. 4, 2022, pp. 694–720. [Online]. Available: https://proceedings.mlsys.org/paper_files/paper/2022/file/6c44dc73014d66ba49b28d483a8f8b0d-Paper.pdf
- [16] T. Jahani-Nezhad, M. A. Maddah-Ali, S. Li, and G. Caire, "Swiftagg: Communication-efficient and dropout-resistant secure aggregation for federated learning with worst-case security guarantees," in *2022 IEEE International Symposium on Information Theory (ISIT)*. IEEE, 2022, pp. 103–108.
- [17] Y. Zhao and H. Sun, "Information theoretic secure aggregation with user dropouts," *IEEE Transactions on Information Theory*, vol. 68, no. 11, pp. 7471–7484, 2022.
- [18] V. R. Cadambe and S. A. Jafar, "Interference alignment and degrees of freedom of the k -user interference channel," *IEEE Transactions on Information Theory*, vol. 54, no. 8, pp. 3425–3441, 2008.
- [19] S. A. Jafar and M. J. Fakhreddin, "Degrees of freedom for the mimo interference channel," *IEEE Transactions on Information Theory*, vol. 53, no. 7, pp. 2637–2642, 2007.
- [20] W. Huang, K. Wan, H. Sun, M. Ji, R. C. Qiu, and G. Caire, "Fundamental limits of distributed linearly separable computation under cyclic assignment," in *2023 IEEE International Symposium on Information Theory (ISIT)*, 2023, pp. 2296–2301.

APPENDIX A
PROOF OF THEOREM 2

By the security constraint in (7), the user can only obtain $W_1 + \dots + W_K$ without accessing any additional information about the messages, even after receiving answers from all servers. Let $X_S = \{X_n : n \in S\}$. From (7), we derive:

$$0 = I(W_1, \dots, W_K; X_{[N]} | W_1 + W_2 + \dots + W_K) \quad (22a)$$

$$= H(X_{[N]} | W_1 + W_2 + \dots + W_K) - H(X_{[N]} | W_1, \dots, W_K) \quad (22b)$$

$$\geq H(X_{[N]} | W_1 + W_2 + \dots + W_K) - H(Q, W_1, \dots, W_K | W_1, \dots, W_K) \quad (22c)$$

$$= H(X_{[N]} | W_1 + W_2 + \dots + W_K) - H(Q) \quad (22d)$$

$$= H(X_{[N]}) - I(X_{[N]}; W_1 + W_2 + \dots + W_K) - H(Q) \quad (22e)$$

$$\geq H(X_{[N]}) - H(W_1 + W_2 + \dots + W_K) - H(Q), \quad (22f)$$

where (22c) follows from the fact that $X_{[N]}$ is a function of Q and $W_1, \dots, W_K$, and (22d) relies on Q being independent of $W_1, \dots, W_K$.

From (22f), we further obtain:

$$\eta L \geq H(Q) \geq H(X_{[N]}) - H(W_1 + \dots + W_K) \quad (23a)$$

$$\geq H(X_{[N]}) - L \quad (23b)$$

$$\geq H(X_{s_1}, \dots, X_{s_{|s|}}) - L \quad (23c)$$

$$\geq \frac{|s|L}{m} - L, \quad (23d)$$

where (23b) follows from the independence of K messages, each uniformly i.i.d. over $[\mathbb{F}_q]^L$.

Next, I will prove (23d) under the symmetric transmission and linear coding. Note that server s_i always contains at least one message W_{s_i} which can not be computed by m servers from $\{s_1, \dots, s_{i-1}\}$. Consider the case that apart from $N - N_r$ stragglers, there are exactly m servers who can compute W_{s_i} . Under the symmetric transmission and linear coding, each server should contain $\frac{1}{m}$ independent part of W_{s_i} . Thus, we have the message of server s_i is independent to messages from $\{s_1, \dots, s_{i-1}\}$, because of the independent part of W_{s_i} . Since the length of each message should be larger than $\frac{L}{m}$, we can obtain (23d) under the symmetric transmission and linear coding.

APPENDIX B
PROOF OF COROLLARY 2

By definition, there are K datasets in the library and we assign $\frac{K}{N}(N - N_r + m)$ datasets to each server. Hence, for any possible assignment, we can find

$$\left\lceil \frac{mK}{M} \right\rceil = \left\lceil \frac{mN}{N - N_r + m} \right\rceil$$

servers, where each server has some dataset which appears at most $m - 1$ times in other $\left\lceil \frac{mN}{N - N_r + m} \right\rceil - 1$ servers. By Theorem 2, we have $\eta^* \geq \frac{\left\lceil \frac{mN}{N - N_r + m} \right\rceil}{m} - 1$.

APPENDIX C
THE GENERAL SCHEME OF THEOREM 3

A. general scheme 1

Data assignment phase. We partition the system into $\lfloor N/M \rfloor$ blocks. For each $i \in [\lfloor N/M \rfloor - 1]$, the i^{th} block includes the datasets $\{D_k : k \in [(i-1)M + 1 : iM]\}$ and the servers in the range $[(i-1)M + 1 : iM]$. The last block includes the datasets and servers in the $[\lfloor N/M \rfloor M + 1 : N]$.

We use the fractional repetition assignment to distribute the data for the first $\lfloor N/M \rfloor - 1$ blocks. As for the last block, it contains $N - \lfloor N/M \rfloor M$ servers and $N - \lfloor N/M \rfloor M$ datasets, with each server being able to compute M datasets. Therefore, we can directly apply the existing data assignment scheme designed for solving the $(N - \lfloor N/M \rfloor M, M)$ non-secure problem to this last block.

Computing phase. For each $i \in [\lfloor N/M \rfloor - 1]$, each server in the i^{th} block computes m linear combinations $\sum_{k \in [(i-1)M + 1 : iM]} W_{k,j}$, and each server sends a random linear combination of these m computed combinations.

Next, we focus on the last block. The computation phase in this block still follows the $(N - \lfloor N/M \rfloor M, M)$ non-secure scheme, which results in a total of $h(N - \lfloor N/M \rfloor M, M)$ linearly independent combinations being sent.

Decoding phase. The user receives information from $N_r = N - M + m$ servers, meaning that responses from $M - m$ servers will not be received. For the first $\lfloor N/M \rfloor - 1$ blocks, each block has M servers, so the user receives at least m servers' responses from each block. Thus, from each block, the user can recover $\sum_{k \in [(i-1)M + 1 : iM]} W_{k,j}$, where j ranges over $[m]$. For the last block, at least $N - \lfloor N/M \rfloor M - M + m$ servers will respond. By carefully designing the coding scheme, we can recover $\sum_{k \in [\lfloor N/M \rfloor M + 1 : N]} W_k$ from the information sent by any $N - \lfloor N/M \rfloor M - M + m$ servers in the last block, together with the transmissions of the first $\lfloor N/M \rfloor - 1$ blocks, the user can recover $W_1 + \dots + W_N$.

In conclusion, we have demonstrated that $h(N, M) = m \lfloor N/M \rfloor + h(N - \lfloor N/M \rfloor M, M)$, which aligns with equation (12). Furthermore, it is evident that $M < N - \lfloor N/M \rfloor M < 2M$ when $N > 2M$ and M does not evenly divide N .

B. general scheme 2

We now focus on the case where $1.5M \leq N < 2M$, and M is even, with $M \geq 2m$. We consider the general Scheme 2 for the (N, M) non-secure problem to prove (13). We assume that a specific feasible scheme already exists for the $(N - M, \frac{M}{2})$ non-secure problem and that this scheme can transmit a total of $h(N - M, \frac{M}{2})$ linearly independent combinations of messages.

Data assignment phase. We first consider the dataset assignment for the servers in $[M]$, which is structured as follows:

server	1	$\dots$	$\frac{M}{2}$	$\frac{M}{2} + 1$	$\dots$	M
dataset	D_1	$\dots$	D_1	D_1	$\dots$	D_1
	$\dots$	$\dots$	$\dots$	$\dots$	$\dots$	$\dots$
	D_y	$\dots$	D_y	D_y	$\dots$	D_y
	D_{y+1}	$\dots$	D_{y+1}	D_{M+1}	$\dots$	D_{M+1}
	$\dots$	$\dots$	$\dots$	$\dots$	$\dots$	$\dots$
	D_M	$\dots$	D_M	D_N	$\dots$	D_N

In this setup, we assign datasets D_1 through D_y to all servers in $[M]$, while each dataset D_k for $k \in [y+1 : N]$ is distributed among $\frac{M}{2}$ servers in $[M]$.

Next, we consider the assignment for the last $N-M$ servers. We need to allocate $N-y = 2(N-M)$ datasets (from $[y+1 : N]$) to a total of $N-M$ servers. Each dataset is assigned to $\frac{M}{2}$ servers, and each server receives M datasets.

Datasets in $[y+1 : N]$ are divided into $\frac{N-y}{2} = N-M$ pairs, where the i^{th} pair is defined as $\mathcal{P}_i = \{y+i, M+i\}$ for $i \in [N-M]$. Thus, we apply the assignment phase of the scheme for the $(N-M, \frac{M}{2})$ non-secure problem, assigning each pair to $\frac{M}{2}$ servers, with each server being allocated $\frac{M}{2}$ pairs.

Computing phase. We divide each message W_k for $k \in [K]$ into m equal-length, non-overlapping sub-messages, denoted as $W_k = \{W_{k,j} : j \in [m]\}$. We first focus on the servers in $[M]$. By constructing the messages sent by these servers, we can recover the following $2m$ linear combinations:

$$F_1 = W_{1,1} + W_{2,1} + \dots + W_{N,1},$$

$\vdots$

$$F_m = W_{1,m} + W_{2,m} + \dots + W_{N,m},$$

$$F_{m+1} = 2(W_{y+1,1} + \dots + W_{M,1}) + W_{M+1,1} + \dots + W_{N,1},$$

$\vdots$

$$F_{2m} = 2(W_{y+1,m} + \dots + W_{M,m}) + W_{M+1,m} + \dots + W_{N,m}.$$

We group the m linear combinations $(F_1 - F_{m+1}), \dots, (F_m - F_{2m})$ into one group, and the m linear combinations $(2F_1 - F_{m+1}), \dots, (2F_m - F_{2m})$ into another group. Since the servers in $[M/2]$ are not assigned any data from $[M+1 : N]$, we let each server in this range compute a random linear combination of the m linear combinations in the first group. The servers in $[M/2+1 : M]$, who are not assigned any data from $[y+1 : M]$, are assigned to compute a random linear combination of the m linear combinations in the second group.

We then focus on the servers in $[M+1 : N]$. For each dataset pair $\mathcal{P}_i = \{y+i, M+i\}$ where $i \in [N-M]$, we define $P_i = 2W_{y+i} + W_{M+i}$. Therefore, we can express F_{m+1} to F_{2m} as $P_{1,1} + \dots + P_{N-M,1}, \dots, P_{1,m} + \dots + P_{N-M,m}$. Thus, we can apply the computing phase of the proposed scheme to the $(N-M, \frac{M}{2})$ non-secure problem.

Decoding phase.

The user receives $N_r = N - M + m$ server responses, which can be categorized into two cases.

First, we consider the case where at least $N - 1.5M + m$ responses come from servers in $[M+1 : N]$. In this scenario, F_{m+1} to F_{2m} can be recovered. Additionally, at least m responses from servers in $[M]$ are received, providing at least m linearly independent combinations. Together with the combinations from F_{m+1} to F_{2m} , this gives a total of $2m$ linearly independent combinations, enabling recovery of F_1 to F_{2m} . We then consider the second case, where the servers in $[M+1 : N]$ respond with $N - 1.5M + m - x$ responses, providing at least $h(N-M, M/2) - x$ linearly independent combinations. Meanwhile, the servers in $[M]$ will have $M/2 + x$ responses, contributing at least $m + x$ linearly independent combinations. In total, these responses provide $h(N-M, M/2)$ linearly independent combinations, which are sufficient to recover F_1 to F_{2m} .

The number of linearly independent combinations transmitted by servers in $[M+1 : N]$ is $h(N-M, \frac{M}{2})$, which spans the space containing F_{m+1} to F_{2m} . Additionally, servers in $[M]$ transmit $2m$ linearly independent combinations, also spanning the space of F_{m+1} to F_{2m} . Therefore, the total number of transmitted linearly independent combinations is $h(N, M) = 2m + h(N-M, \frac{M}{2}) - m = h(N-M, \frac{M}{2}) + m$, which matches (13).

C. general scheme 3

We now consider the (N, M) non-secure problem, where $1.5M \leq N < 2M$, M is odd, and $M \geq 2m + 1$. We aim to construct a scheme (Scheme 3) to prove (14). We also define that $N = 2M - y$.

Data assignment phase. The assignment is shown at the top of the next page. In this assignment, we split the N datasets into three sections. The first section includes $D_1, \dots, D_t$ (The reason for choosing $t = \frac{M-1}{2}$ is provided in [12].), and these datasets are assigned to servers in $[M]$. The second section, containing $D_{t+1}, \dots, D_M$, is assigned to servers in $[y] \cup [M+1 : N]$. The third section, consisting of $D_{M+1}, \dots, D_N$, is cyclically allocated to servers in $[y+1 : M]$, where each server receives $M-t$ adjacent datasets within $[\frac{M-1}{2} + 1 : N]$. Additionally, datasets $D_{M+1}, \dots, D_N$ are assigned to servers in $[M+1 : N]$ in a cyclic manner such that each server receives t consecutive datasets within $[\frac{M-1}{2} + 1 : N]$.

Computing phase. We first divide each message W_k for $k \in [K]$ into m equal-length, non-overlapping sub-messages, denoted as $W_k = \{W_{k,j} : j \in [m]\}$. We design the computing phase so that the total number of linearly independent combinations computed by all servers is $\frac{M+1}{2} - y + 2m$. We let each server send a linear combination of messages so that the user can recover $\mathbf{F}' [W_{1,1}; \dots; W_{N,m}]$ from the responses of any $N_r = N - M + m$ servers, where $\mathbf{F}'$ is shown at the top of the next page in (24).

server	1	$\dots$	y	$y + 1$	$y + 2$	$\dots$	M	M+1	M + 2	$\dots$	N
dataset	D_1	$\dots$	D_1	D_1	D_1	$\dots$	D_1	$D_{\frac{M-1}{2}+1}$	$D_{\frac{M-1}{2}+1}$	$\dots$	$D_{\frac{M-1}{2}+1}$
	$\dots$	$\dots$	$\dots$	$\dots$	$\dots$	$\dots$	$\dots$	$\dots$	$\dots$	$\dots$	$\dots$
	$D_{\frac{M-1}{2}}$	$\dots$	$D_{\frac{M-1}{2}}$	$D_{\frac{M-1}{2}}$	$D_{\frac{M-1}{2}}$	$\dots$	$D_{\frac{M-1}{2}}$	D_M	D_M	$\dots$	D_M
	$D_{\frac{M-1}{2}+1}$	$\dots$	$D_{\frac{M-1}{2}+1}$	D_{M+1}	D_{M+2}	$\dots$	D_N	D_{M+1}	D_{M+2}	$\dots$	D_N
	$\dots$	$\dots$	$\dots$	$\dots$	$\dots$	$\dots$	$\dots$	$\dots$	$\dots$	$\dots$	$\dots$
	D_M	$\dots$	D_M	$D_{\frac{3M+1}{2}}$	$D_{\frac{3M+1}{2}+1}$	$\dots$	$D_{\frac{M-1}{2}-1}$	$D_{\frac{3M-1}{2}}$	$D_{\frac{3M-1}{2}+1}$	$\dots$	$D_{\frac{M-1}{2}-1}$

$$\mathbf{F}' = \begin{bmatrix} \mathbf{f}_1 \\ \vdots \\ \mathbf{f}_{\frac{M+1}{2}-y+2m} \end{bmatrix} = \begin{bmatrix} (\mathbf{F})_{2 \times N} & \mathbf{0}_{2 \times N} & \cdots & \mathbf{0}_{2 \times N} \\ \mathbf{0}_{2 \times N} & (\mathbf{F})_{2 \times N} & \cdots & \mathbf{0}_{2 \times N} \\ \vdots & \vdots & \ddots & \vdots \\ \mathbf{0}_{2 \times N} & \mathbf{0}_{2 \times N} & \cdots & (\mathbf{F})_{2 \times N} \\ (\mathbf{V}_1)_{(\frac{M+1}{2}-y) \times N} & (\mathbf{V}_2)_{(\frac{M+1}{2}-y) \times N} & \cdots & (\mathbf{V}_m)_{(\frac{M+1}{2}-y) \times N} \end{bmatrix}_{(\frac{M+1}{2}-y+2m) \times Nm}. \quad (24)$$

We design the demand matrix

$$\mathbf{F} = \begin{bmatrix} \boxed{\begin{matrix} 1, \dots, 1 \\ 0, \dots, 0 \end{matrix}} & \boxed{\begin{matrix} 1, \dots, 1 \\ 2, \dots, 2 \end{matrix}} & \boxed{\begin{matrix} 1, \dots, 1 \\ 1, \dots, 1 \end{matrix}} \end{bmatrix} \quad (25)$$

We divide matrix $\mathbf{F}$ into three column-wise sub-matrices: $\mathbf{C}_1$, containing $\frac{M-1}{2}$ columns, corresponding to the messages in $[\frac{M-1}{2}]$, $\mathbf{C}_2$, containing $\frac{M+1}{2}$ columns, corresponding to the messages in $[\frac{M+1}{2} : M]$, and $\mathbf{C}_3$, containing $(N-M)$ columns, corresponding to the messages in $[M+1 : N]$.

Let the matrix $\mathbf{V}_i$, for $i \in [m]$, be defined as

$$\mathbf{V}_i = \begin{bmatrix} \boxed{0, \dots, 0} & \boxed{0, \dots, 0} & \boxed{*, \dots, *} \\ \vdots & \vdots & \vdots \\ \boxed{0, \dots, 0} & \boxed{0, \dots, 0} & \boxed{*, \dots, *} \end{bmatrix} \quad (26)$$

In the matrix $\mathbf{V}_i$, the entries denoted by $*$ are the unknowns we need to design. And $\mathbf{C}_1$, $\mathbf{C}_2$, and $\mathbf{C}_3$ have column counts consistent with the definitions above.

Now, we focus on the servers in $[y]$, which have the same datasets and do not contain datasets from $[M + 1 : N]$. From these servers, we aim to recover:

$$\begin{aligned} F_1 - F_2 &= W_{1,1} + \cdots + W_{t,1} - W_{t+1,1} - \cdots - W_{N,1}, \\ &\vdots \end{aligned}$$

$$F_{2m-1} - F_{2m} = W_{1,m} + \cdots + W_{t,m} - W_{t+1,m} - \cdots - W_{N,m}.$$

Thus, we let the servers in $[y]$ each compute a random linear combination of the m linearly independent combinations $(F_1 - F_2), \dots, (F_{2m-1} - F_{2m})$.

For the servers in $[M + 1 : N]$ that do not contain datasets from $[t]$, we design their transmissions so that the user can recover $F_2, F_4, \dots, F_{2m}, F_{2m+1}, \dots, F_{\frac{M+1}{2}-y+2m}$ from any $\frac{M+1}{2} - y + m$ responses. For $n \in [M + 1, N]$, we let server n compute

$$S_n[F_2; F_4; \dots; F_{2m}; F_{2m+1}; \dots; F_{\frac{M+1}{2}-y+2m}]$$

$$\times [W_{1,1}; W_{2,1}; \dots; W_{N,m}]. \quad (27)$$

We design $\mathbf{s}_n$ as follows. Notice that $W_1, \dots, W_{\frac{M-1}{2}}$ can be computed by server n ; and that in the linear combination (27), the coefficients of $W_{\frac{M-1}{2}+1}, \dots, W_M$ are 0. Hence, to guarantee that in (27) the coefficients of the messages which server n cannot compute are 0, we only need to consider the messages in $W_{M+1}, \dots, W_N$, corresponding to the columns in $\mathbf{C}_3$ in the matrix, with a total of $m(N - M)$ columns. By extracting the corresponding rows and columns from the matrix $\mathbf{F}'$, we obtain $\mathbf{F}_s$, as shown at the top of the next page in (28), where $\mathbf{V}'_i$ is the sub-matrix formed by the last $N - M$ columns of $\mathbf{V}_i$. Notice that the datasets in $[M + 1 : N]$ are placed in a cyclic assignment among the servers in $[M + 1 : N]$. Each server cannot compute $N - M - \frac{M-1}{2}$ of the datasets in $[M + 1 : N]$. By extracting the corresponding columns from the matrix $\mathbf{F}_s$, we form a submatrix $\mathbf{F}_{s_n}$, with dimensions $(\frac{M+1}{2} - y + m) \times m(N - \frac{3M-1}{2})$. Our desired vector $\mathbf{s}_n$ is then the left null vector of this matrix. If the number of rows in the matrix $\mathbf{F}_{s_n}$ is greater than the number of columns, we can directly assign random values to the entries denoted by $*$ as in [12] to obtain $\mathbf{s}_n$. If the number of rows is less than or equal to the number of columns, we apply the interference alignment strategy from [20] to design the values of the $*$ entries in order to obtain $\mathbf{s}_n$.

Finally, we focus on the servers in $[y + 1 : M]$. We design their transmissions so that the user can recover $2F_1 - F_2, 2F_3 - F_4, \dots, 2F_{2m-1} - F_{2m}, F_{2m+1}, \dots, F_{\frac{M+1}{2}-y+2m}$ from any $\frac{M+1}{2} - y + m$ responses. For $n \in [y + 1, M]$, we also let server n compute

$$\begin{aligned} & s_n[2F_1 - F_2; \dots; 2F_{2m-1} - F_{2m}; F_{2m+1}; \dots; F_{\frac{M+1}{2}-y+2m}] \\ & \times [W_{1,1}; W_{2,1}; \dots; W_{N,m}]. \end{aligned} \quad (29)$$

We design s_n as follows. Notice that $W_1, \dots, W_{\frac{M-1}{2}}$ can be computed by server n ; and that in the linear combination (29) the coefficients of $W_{\frac{M-1}{2}+1}, \dots, W_M$ are 0. Hence, in order to guarantee that in (29) the coefficients of the messages which server n cannot compute are 0, we only need to

$$\mathbf{F}_s = \begin{bmatrix} \mathbf{1}_{1 \times (N-M)} & \mathbf{0}_{1 \times (N-M)} & \cdots & \mathbf{0}_{1 \times (N-M)} \\ \mathbf{0}_{1 \times (N-M)} & \mathbf{1}_{1 \times (N-M)} & \cdots & \mathbf{0}_{1 \times (N-M)} \\ \vdots & \vdots & \ddots & \vdots \\ \mathbf{0}_{1 \times (N-M)} & \mathbf{0}_{1 \times (N-M)} & \cdots & \mathbf{1}_{1 \times (N-M)} \\ (\bar{\mathbf{V}}'_1)_{(\frac{M+1}{2}-y) \times (N-M)} & (\bar{\mathbf{V}}'_2)_{(\frac{M+1}{2}-y) \times (N-M)} & \cdots & (\bar{\mathbf{V}}'_m)_{(\frac{M+1}{2}-y) \times (N-M)} \end{bmatrix}_{(\frac{M+1}{2}-y+m) \times m(N-M)} \quad (28)$$

$$\mathbf{F}'_s = \begin{bmatrix} \mathbf{1}_{1 \times (N-M)} & \mathbf{0}_{1 \times (N-M)} & \cdots & \mathbf{0}_{1 \times (N-M)} \\ \mathbf{0}_{1 \times (N-M)} & \mathbf{1}_{1 \times (N-M)} & \cdots & \mathbf{0}_{1 \times (N-M)} \\ \vdots & \vdots & \ddots & \vdots \\ \mathbf{0}_{1 \times (N-M)} & \mathbf{0}_{1 \times (N-M)} & \cdots & \mathbf{1}_{1 \times (N-M)} \\ (\bar{\mathbf{V}}'_1)_{(\frac{M+1}{2}-y) \times (N-M)} & (\bar{\mathbf{V}}'_2)_{(\frac{M+1}{2}-y) \times (N-M)} & \cdots & (\bar{\mathbf{V}}'_m)_{(\frac{M+1}{2}-y) \times (N-M)} \end{bmatrix}_{(\frac{M+1}{2}-y+m) \times m(N-M)} \quad (30)$$

consider the messages in $W_{M+1}, \dots, W_N$, corresponding to the columns in $\mathbf{C}_3$ in the matrix, with a total of $m(N-M)$ columns. By extracting the corresponding rows and columns from the matrix $\mathbf{F}'$, we obtain $\mathbf{F}'_s$, as shown at the top of the next page in (30). It is easy to observe that $\mathbf{F}'_s = \mathbf{F}_s$. Notice that the datasets in $[M+1, N]$ are also cyclically assigned to the servers in $[y+1 : M]$. For $n \in [y+1 : M]$, server n cannot compute $N - \frac{3M+1}{2}$ datasets from $[M+1, N]$, and these datasets are also not computable by server $n + M - y$. Since $\mathbf{F}'_s = \mathbf{F}_s$, we can set $s_n = s_{n+M-y}$.

Decoding phase. The user receives responses from $N_r = N - M + m$ servers, of which x responses are from servers in $[y]$. Depending on the value of x , we can categorize the scenarios into the following two cases:

- *Case 1:* $x \leq m$. In this case, the user receives x linearly independent combinations sent by the servers in $[y]$. The user still needs to collect an additional $2m - x + N - \frac{3M-1}{2}$ linearly independent combinations from any $(N - M - x + m)$ servers in $[y+1 : N]$. In the worst-case scenario, the servers in one of the groups, either $[y+1 : M]$ or $[M+1 : N]$, all respond, transmitting a total of $m + N - \frac{3M-1}{2}$ linearly independent combinations. Additionally, $m - x$ servers from the other group respond, providing another $m - x$ linearly independent combinations. Together, this results in a total of $2m + N - \frac{3M-1}{2}$ linearly independent combinations, sufficient to fully recover the demand matrix $\mathbf{F}'$.
- *Case 2:* $m < x \leq y$. In this case, the user receives m linearly independent combinations sent by the servers in $[y]$. We consider the worst-case scenario, where $x = y$, meaning that the user still needs to collect an additional $m + N - \frac{3M-1}{2}$ linearly independent combinations from any $2N - 3M + m$ servers in $[y+1 : N]$. Since $N \geq \frac{3M+1}{2}$, it follows that $2N - 3M + m \geq m + N - \frac{3M-1}{2}$. Additionally, any $m + N - \frac{3M-1}{2}$ servers in either of the groups $[y+1 : M]$ or $[M+1 : N]$ compute linearly independent combinations. Therefore, it is possible to receive $m + N - \frac{3M-1}{2}$ linearly independent combinations from any $2N - 3M + m$ servers in $[y+1 : N]$.

Thus, the user obtains a total of $2m + N - \frac{3M-1}{2}$ linearly independent combinations, which suffice to reconstruct the demand matrix $\mathbf{F}'$.

By the above scheme, the number of linearly independent transmissions by all servers is equal to the number of rows in $\mathbf{F}'$, i.e., $\frac{M+1}{2} - y + 2m = N - \frac{3M-1}{2} + 2m$, coinciding with (14).

D. general scheme 4

In this section, we examine the case where $M < N < 1.5M$, with $M \geq 2m$, and construct a recursive scheme (Scheme 4) that aims to demonstrate (15). This recursive scheme builds on the solution for the $(M, 2M-N)$ non-secure problem, assuming that this solution has been previously developed, and provides a total of $h(M, 2M-N)$ linearly independent combinations of messages.

Data assignment phase. We begin by allocating datasets $D_1, \dots, D_{N-M}$ to each server in $[M]$. Subsequently, datasets $D_{N-M+1}, \dots, D_N$ are assigned to each server in $[M+1 : N]$, ensuring that each server in $[M+1 : N]$ has access to M datasets, while each server in $[M]$ holds fewer than M datasets, specifically $N-M$. Additionally, each dataset in $[N-M+1 : N]$ is assigned to $N-M < M$ servers. Therefore, in the next step, we assign each dataset D_k for $k \in [N-M+1 : N]$ to $2M-N$ servers in $[M]$, such that each server in $[M]$ obtains $2M-N$ datasets in $[N-M+1 : N]$. This allocation follows the assignment phase of the proposed scheme for the $(M, 2M-N)$ non-secure problem.

Computing phase. Each message W_k for $k \in [K]$ is divided into m equal-length, non-overlapping sub-messages, noted as $W_k = \{W_{k,j} : j \in [m]\}$. First, we focus on the $(M, 2M-N)$ non-secure problem, where the messages are defined as $W'_1, \dots, W'_M$. Under this scheme, each server computes a linear combination of these M messages. Letting the total number of linearly independent combinations across all servers be $h(M, 2M-N)$, we can represent these $h(M, 2M-N)$ combinations as

$$\mathbf{F}' [W'_{1,1}; \dots; W'_{M,m}]. \quad (31)$$

Each transmission from a server $n' \in [M]$ is represented as

$$s_{n'} \mathbf{F}' [W'_{1,1}; \dots; W'_{M,m}].$$

Returning to the (N, M) non-secure problem, we set up the server transmissions so that, together, they produce $h(M, 2M-N)$ linearly independent combinations, represented as $\mathbf{F} [W_{1,1}; \dots; W_{N,m}]$, where $\mathbf{F}$ is shown at the top of the next

$$\mathbf{F} = \begin{bmatrix} \text{---} \mathbf{1}_{1 \times (N-M)} \text{---} & \text{---} \mathbf{1}_{1 \times M} \text{---} & \text{---} \mathbf{0}_{1 \times (N-M)} \text{---} & \text{---} \mathbf{0}_{1 \times M} \text{---} & \cdots & \text{---} \mathbf{0}_{1 \times (N-M)} \text{---} & \text{---} \mathbf{0}_{1 \times M} \text{---} \\ \text{---} \mathbf{0}_{1 \times (N-M)} \text{---} & \text{---} \mathbf{1}_{1 \times M} \text{---} & \text{---} \mathbf{1}_{1 \times (N-M)} \text{---} & \text{---} \mathbf{1}_{1 \times M} \text{---} & \cdots & \text{---} \mathbf{0}_{1 \times (N-M)} \text{---} & \text{---} \mathbf{0}_{1 \times M} \text{---} \\ \vdots & \vdots & \vdots & \vdots & \ddots & \vdots & \vdots \\ \text{---} \mathbf{0}_{1 \times (N-M)} \text{---} & \text{---} \mathbf{0}_{1 \times M} \text{---} & \text{---} \mathbf{0}_{1 \times (N-M)} \text{---} & \text{---} \mathbf{0}_{1 \times M} \text{---} & \cdots & \text{---} \mathbf{1}_{1 \times (N-M)} \text{---} & \text{---} \mathbf{1}_{1 \times M} \text{---} \end{bmatrix}_{h(M, 2M-N-m) \times (N-M) \quad h(M, 2M-N-m) \times M \quad \cdots \quad h(M, 2M-N-m) \times (N-M) \quad h(M, 2M-N-m) \times M} \quad (32)$$

page in (32). Each row in the submatrix $\mathbf{A}$ contains identical random numbers. For each $i \in [m]$, $\mathbf{V}_i$ shares elements with the corresponding elements in matrix $\mathbf{F}$, allowing matrix $\mathbf{F}$ to be viewed as a submatrix extracted from $\mathbf{F}'$ based on the columns in each $\mathbf{V}_i$.

For each server $n \in [M]$, by the construction in the assignment phase, it has access to datasets $D_1, \dots, D_{N-M}$, while the assignment for datasets $D_{N-M+1}, \dots, D_N$ is based on the assignment for the $(M, 2M-N)$ non-secure problem. Thus, server n computes $s_n \mathbf{F} [W_1; \dots; W_N]$, where s_n corresponds to the transmission vector of server n in the $(M, 2M-N)$ problem.

For each server $n \in [M+1 : N]$, it cannot compute $W_1, \dots, W_{N-M}$, which corresponds to the columns of submatrix $\mathbf{A}$ in $\mathbf{F}$. Extracting these columns creates a matrix $\mathbf{F}''$ of size $h(M, 2M-N) \times m(N-M)$, whose rank is m . Consequently, the left null space of $\mathbf{F}''$ includes $h(M, 2M-N) - m$ independent vectors. We let the transmission vector s_n for each server n be a random linear combination of these vectors, using uniformly i.i.d. coefficients from $\mathbb{F}_q$, so that each server n computes $s_n \mathbf{F} [W_1; \dots; W_N]$.

Decoding phase. Suppose $\mathcal{A}$ denotes the set of responding servers with $|\mathcal{A}| = N_r$, $\mathcal{A}_1 = \mathcal{A} \cap [M]$, and $\mathcal{A}_2 = \mathcal{A} \setminus [M]$, where $|\mathcal{A}_2| \leq N-M = N_r - m$. When $M \geq N-M+m = N_r$, if $|\mathcal{A}_2| = 0$, the user can recover the first m rows of $\mathbf{F}$, leveraging the decodability of the $(M, 2M-N)$ scheme.

When $0 < |\mathcal{A}_2| \leq N-M$ and $\mathcal{A}_1$ alone does not suffice, the user receives at least m independent responses from $\mathcal{A}_1$. Let the linearly independent combinations from $\mathcal{A}_1$ be denoted by λ_1 . In addition to the responses from $\mathcal{A}_1$, the user will receive responses from at least $h(M, 2M-N) - \lambda_1$ servers in $[M+1 : N]$. Since each server sends an independent combination of $h(M, 2M-N) - m$, any $h(M, 2M-N) - \lambda_1$ responses in $[M+1 : N]$ are linearly independent with high probability. Combining with λ_1 , this totals $h(M, 2M-N)$ combinations, allowing the user to recover $\mathbf{F} [W_{1,1}; \dots; W_{N,m}]$.

In summary, $h(N, M) = h(M, 2M-N)$, consistent with (15).

APPENDIX D

THE APPLICABLE RANGE OF INTERFERENCE ALIGNMENT IN GENERAL SCHEME 3

Recall that in (28), $\mathbf{F}_s$ is a matrix of dimensions $(\frac{M+1}{2} - y + m) \times m(N-M)$. We use $N-M$ workers to compute this required matrix, where each worker can compute $\frac{M-1}{2}$ datasets. If we proceed with interference alignment individually for each worker as in Example 1, we should generate $(m-1)(N - \frac{3M}{2} - \frac{1}{2})$ linearly independent linear equations for each worker. Thus, in total, we generate $(N-M)(m-1)(N - \frac{3M}{2} - \frac{1}{2})$ linear equations. The resulting matrix $\mathbf{E}$ has

dimensions $[(N-M)(m-1)(N - \frac{3M}{2} - \frac{1}{2})] \times m(N-M)$. By construction, these $(N-M)(m-1)(N - \frac{3M}{2} - \frac{1}{2})$ linear equations are linearly independent, and thus, $\mathbf{E}$ is of full rank. The left null space of $\mathbf{E}^T$ contains only $m(N-M) - (N-M)(m-1)(N - \frac{3M}{2} - \frac{1}{2})$ linearly independent vectors. However, the number of rows of $\mathbf{F}_s$ is $(\frac{M+1}{2} - y + m) = N - \frac{3M}{2} + \frac{1}{2} + m$. Therefore, when

$$m(N-M) - (N-M)(m-1) \left(N - \frac{3M}{2} - \frac{1}{2} \right) < N - \frac{3M}{2} + \frac{1}{2} + m,$$

i.e.,

$$m(N-M-1) \leq [(N-M)(m-1) + 1] \left(N - \frac{3M}{2} - \frac{1}{2} \right),$$

we need to use the interference alignment computing scheme, which jointly aligns the interferences across different workers.